

《区块链基础及应用》实验报告

Ex1: 比特币测试网 P2PKH 交易

姓名: 邹俊轩 学号: 2412592

实验日期: 2026 年 9 月 28 日

1 项目构成与实验目标

项目 GitHub 仓库: https://github.com/pointzu/blockchain_experiment_ex1。源码、交易记录、截图及本报告均整理于该仓库。

本实验依据课程 Ex1.pdf 和提交要求, 使用 Python 构造 Bitcoin testnet3 交易。在领取测试币后, 将一个未花费输出拆分为十个输出, 再通过显式编写的 P2PKH 锁定脚本与解锁脚本, 花费其中一份并发送至课程配置提供的地址。代码说明、金额和结果均对应本次实际实验。

1.1 项目文件

文件	本次用途与修改
config.py	从本地文件或环境变量读取私钥, 派生公钥和地址并核对领取记录。
keygen.py	预备实验生成测试网私钥和地址;
split_test_coins .py	设置领取交易、输出编号、总输出金额及份数, 构造十个独立输出并签名。
ex1.py	补全两个 P2PKH 函数, 花费分币交易的第 0 号输出。
utils.py	金额转整数 satoshi、本地脚本验证、链上输入检查、广播和哈希保存。
runtime_setup.py	为当前 Python 进程优先使用已有的同位数 OpenSSL DLL。
transactions.py	保存领取、分币和发币的真实交易哈希。
check_results.py	通过公开 API 查询交易确认状态、输入与输出。
requirements.txt	python-bitcoinlib 与 requests 依赖。

1.2 提交材料与完成标准

提交要求包括填好的脚本源代码、交易输出结果和相关截图。对应保存 split_output.txt、ex1_output.txt 和领取、分币、发币截图; chain_status.txt 为最终确认状态的补充记录。以脚本验证通过、广播成功且链上交易已确认作为完成标准。

2 实验原理

2.1 密钥、地址与 UTXO

私钥用于签名，公钥用于验证签名。本实验使用的传统 P2PKH 地址由公钥的 Hash160 值结合测试网版本信息和校验码编码得到。数字签名提供花费授权；链上数据和收款地址则可以公开查询。

比特币通过未花费交易输出（UTXO）表示可花费资金。一个输入用“上一笔交易的哈希 + 输出编号”定位旧输出。编号从 0 开始，与用户做过多少次交易无关。旧输出被整体花费，新交易创建新的输出；同一个旧输出不能被重复花费。

2.2 P2PKH 脚本执行

锁定脚本 `scriptPubKey` 指定花费条件，解锁脚本 `scriptSig` 提供签名和公钥：

```
scriptSig:    <signature> <public_key>
scriptPubKey: OP_DUP OP_HASH160 <public_key_hash>
              OP_EQUALVERIFY OP_CHECKSIG
```

节点在验证旧输出的花费时执行两部分脚本。下表右侧为栈顶：

步骤	栈内容	作用
解锁材料入栈	签名、公钥	提交签名和公钥。
OP_DUP	签名、公钥、公钥	复制公钥，保留一份用于验签。
OP_HASH160	签名、公钥、计算出的哈希	对公钥执行 SHA-256 后再执行 RIPEMD-160。
压入预期哈希	签名、公钥、计算出的哈希、预期哈希	提供旧输出中指定的公钥哈希。
OP_EQUALVERIFY	签名、公钥	比较哈希；相同则移除二者并继续，不同则失败。
OP_CHECKSIG	验证成功时为 True	用公钥验证本笔交易的签名。

2.3 金额与手续费

交易输出金额以整数 satoshi（sat）表示， $1 \text{ BTC} = 10^8 \text{ sat}$ 。本实验的金额关系为：

$$\text{输入总额} = \text{输出总额} + \text{手续费}$$

脚本变量 `amount_to_send` 表示输出金额，并非手续费。若未创建找零输出，输入与输出的差额不会自动退回，而是全部成为手续费。本实验先分币再花其中一份，以保留其他九份供后续实验使用。

3 实验步骤与实现

3.1 环境配置与领取测试币

实验在 Windows 和 Python 3.13 环境中运行，依赖 `python-bitcoinlib` 与 `requests`。曾因 Python 加载其他软件目录中的不匹配 DLL 出现 `WinError 193`；通过 `runtime_setup.py` 在当前进程的搜索路径中优先放置 `ObtainTestcoin` 目录，并核对 DLL 位数后，钱包模块正常导入。

将预备实验保留的原私钥写入本地配置后，派生地址与领取截图中的地址一致：

`mzT3NxNH1iGLvKiHDzJyN4NtLogEFLv3A2`

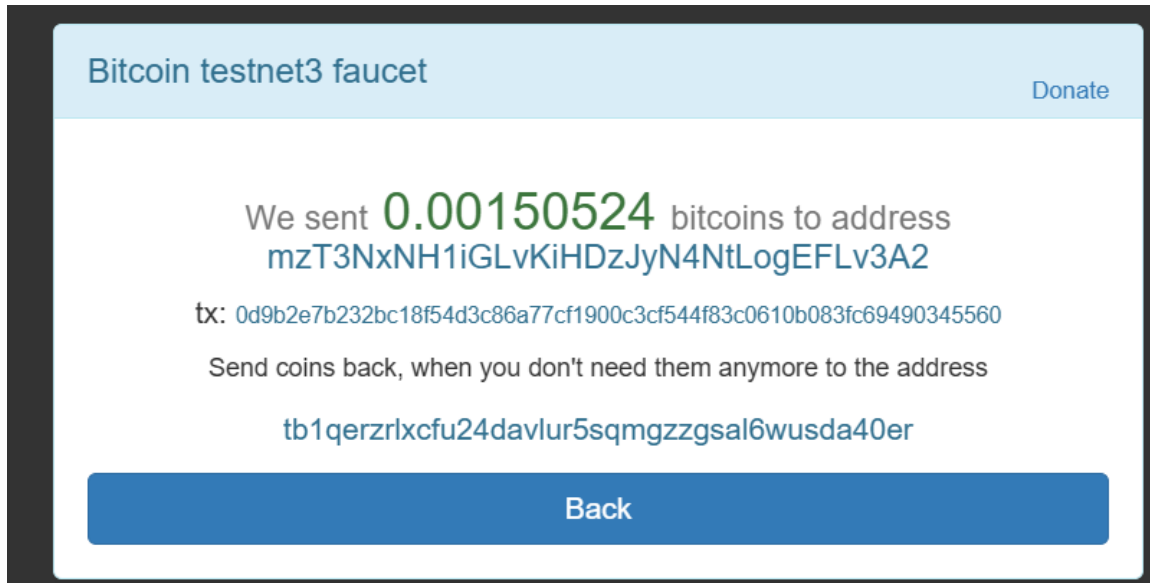


图 1：水龙头领取成功记录：0.00150524 BTC

领取交易哈希为：

`0d9b2e7b232bc18f54d3c86a77cf1900c3cf544f83c0610b083fc69490345560`

公开 API 的查询结果表明，该交易的第 1 号输出属于本人地址，金额为 **150524 sat**，确认区块为 **5148688**。因此分币使用 `utxo_index=1`。

课程原示例使用 BlockCypher。实验期间该接口查不到领取交易；根据助教提示，使用 testnet3 的 mempool 与 Blockstream 查询。广播实现另行改为 Blockstream 的 Esplora `POST /tx` 接口，请求正文直接包含十六进制原始交易，成功返回交易哈希。查询网站替换和广播接口改动是两项不同的调整。

3.2 构造并签名分币交易

为后续练习保留多个独立输出，设置 $n=10$ 。分币输入使用领取交易的第 1 号输出，所有新输出仍锁定到本人地址。关键参数为：

```
amount_to_send = Decimal('0.00149')
txid_to_spend = (
    '0d9b2e7b232bc18f54d3c86a77cf1900c3cf544f83c0610b083fc69490345560')
utxo_index = 1
n = 10
```

分币交易的关键构造与签名代码如下（摘录自完成的脚本）：

```
txin_scriptPubKey = my_address.to_scriptPubKey()
txin = create_txin(txid_to_spend, utxo_index)
each_amount = Decimal(str(amount_to_send)) / n
tx = CMutableTransaction(
    [txin],
    [create_txout(each_amount, txin_scriptPubKey)
     for _ in range(n)])
sighash = SignatureHash(txin_scriptPubKey, tx, 0, SIGHASH_ALL)
txin.scriptSig = CScript([
    my_private_key.sign(sighash) + bytes([SIGHASH_ALL]),
    my_public_key])
VerifyScript(txin.scriptSig, txin_scriptPubKey,
             tx, 0, (SCRIPT_VERIFY_P2SH,))
```

这里针对整笔十输出交易计算签名摘要。SIGHASH_ALL 承诺交易的所有输出，使签名与本次付款内容关联。代码为每一份创建独立的 CMutableTxOut 对象。

utils.py 中采用 Decimal 计算 satoshi，检查金额没有不足 1 sat 的部分后转为整数，避免浮点误差。链上预检查还验证输入交易已确认、输出属于本地地址且尚未花费。

$$150524 = 10 \times 14900 + 1524 \quad (\text{sat}).$$

本次实际交易大小为 **497 bytes**，手续费率为 $1524/497 \approx 3.07 \text{ sat/vB}$ 。这些数值来源于实际广播输出。

3.3 分币运行结果与链上确认

先以默认预览模式检查金额和签名，确认正确后通过显式 `--broadcast` 参数提交交易：

```
python '.\split_test_coins .py' --broadcast 2>&1 |
Tee-Object split_output.txt
```

```
PS C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1> python '.\split_test_coins .py' --broadcast 2>&1 | Tee-Object split_output.txt
My address: mzt3NxnH1iGLvKiHDzJyN4NtLogEFLv3A2
Spending: 0d9b2e7b232bc18f54d3c86a77cf1900c3cf544f83c0610b083fc69490345560 vout= 1
Network: Bitcoin testnet3
Input: 150524 sat
Outputs: 10; total: 149000 sat
  output[0]: 14900 sat
  output[1]: 14900 sat
  output[2]: 14900 sat
  output[3]: 14900 sat
  output[4]: 14900 sat
  output[5]: 14900 sat
  output[6]: 14900 sat
  output[7]: 14900 sat
  output[8]: 14900 sat
  output[9]: 14900 sat
Fee: 1524 sat; size: 497 bytes; fee rate: 3.07 sat/vB
Local txid: 86c6408a8b95ca4bd7fa71c3f2ef362016d3fc4ef01e59fc62742d1c5eeae2d6
VerifyScript: PASS
HTTP: 200 OK
86c6408a8b95ca4bd7fa71c3f2ef362016d3fc4ef01e59fc62742d1c5eeae2d6
Saved: C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1\transactions.py
Explorer: https://blockstream.info/testnet/tx/86c6408a8b95ca4bd7fa71c3f2ef362016d3fc4ef01e59fc62742d1c5eeae2d6
Broadcast accepted. Wait for Confirmed before taking the final explorer screenshot.
PS C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1>
```

图 2：分币脚本输出：十份各 14900 sat，脚本验证通过且 HTTP 200

广播返回的分币交易哈希为：

86c6408a8b95ca4bd7fa71c3f2ef362016d3fc4ef01e59fc62742d1c5eeae2d6

随后通过交易浏览器检查输出列表和确认状态。交易最初显示 **Unconfirmed**，说明已传播但尚未入块；等待后获得确认，所在区块为 **5151331**。只有链上确认才说明本次分币交易已经被收录，**HTTP 200** 本身不能替代确认。

分币后，领取交易的原输出已被消费，新的十个输出编号为 0 至 9，均属于本人地址。后续 Ex1 必须引用这笔新交易的哈希，不能再次引用已花费的领取输出。

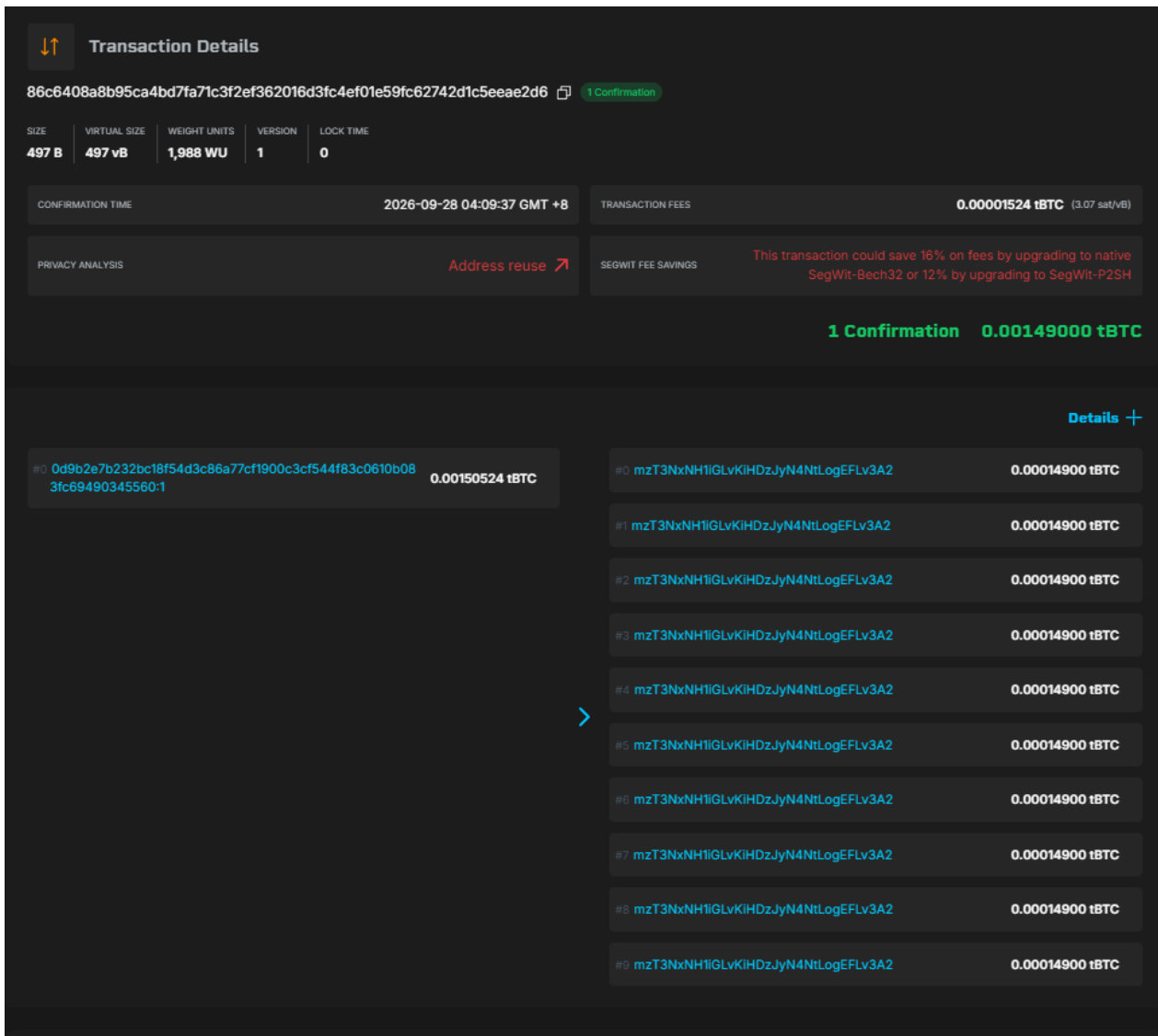


图 3: 分币交易已确认: 十个输出均为本人地址, 每份 0.000149 BTC

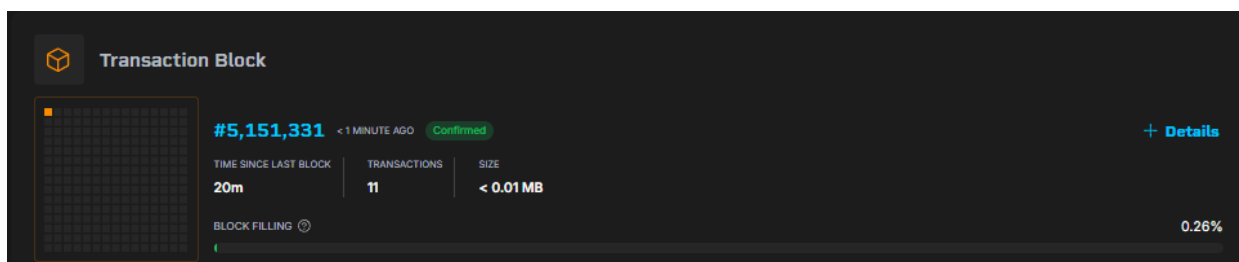


图 4: 分币交易被收录至区块 5151331

3.4 补全 P2PKH 锁定脚本与解锁脚本

ex1.py 的两个核心 TODO 按课程要求在操作码级别完成，没有用 `to_scriptPubKey()` 直接替代锁定脚本实现：

```
def P2PKH_scriptPubKey(address):
    if not isinstance(address, P2PKHBitcoinAddress):
        raise ValueError('P2PKH address required')
    return [OP_DUP, OP_HASH160, bytes(address),
            OP_EQUALVERIFY, OP_CHECKSIG]

def P2PKH_scriptSig(txin, txout, txin_scriptPubKey):
    from config import my_private_key, my_public_key
    signature = create_OP_CHECKSIG_signature(
        txin, txout, txin_scriptPubKey, my_private_key)
    return [signature, my_public_key]
```

以上摘录仅将报错提示译为英文，便于排版，核心实现与实际脚本相同。`bytes(address)` 获取 P2PKH 地址解码后的 20 字节公钥哈希，而不是地址字符串的字符编码。

签名辅助函数对待发送交易计算签名摘要，并在签名后附加 `SIGHASH_ALL` 字节。`scriptSig` 的两个元素依次为签名、公钥，对应上一节介绍的栈执行顺序。

发币输入选取分币交易的第 0 号输出，金额为 **14900 sat**。这里的 0 表示分币交易的首个输出，不表示“第一次交易”。主程序从 `transactions.py` 读取成功广播的分币哈希，并设置：

```
amount_to_send = Decimal('0.000139')
utxo_index = 0
txid_to_spend = args.txid or split_txid
```

目标输出锁定到课程配置中的收款地址：

`mv4rnyY3Su5gjcDNzbMLKBQkBicCtHUtFB`

$$14900 = 13900 + 1000 \quad (\text{sat}).$$

代码在本地校验通过后才允许广播。该过程用本人的签名解锁旧输出，同时为收款方创建新的锁定输出；收款方后续花费这笔资金时需要其自己的私钥。

3.5 发币运行结果

分币确认后，先运行 `python ex1.py` 检查预览，再实际广播并保存终端输出：

```
python '.\ex1.py' --broadcast 2>&1 |
Tee-Object ex1_output.txt
```

```
PS C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1> python '.\ex1.py' --broadcast 2>&1 | Tee-Object ex1_output
.txt
My address: mzT3NxNH1iGLvKiHDzJyN4NtLogEFLv3A2
Recipient: mv4rnyY3Su5gjcDNzbMLKBQk8icCtHutFB
Spending: 86c6408a8b95ca4bd7fa71c3f2ef362016d3fc4ef01e59fc62742d1c5eeae2d6 vout= 0
Network: Bitcoin testnet3
Input: 14900 sat
Outputs: 1; total: 13900 sat
  output[0]: 13900 sat
Fee: 1000 sat; size: 192 bytes; fee rate: 5.21 sat/vB
Local txid: c49a4a8c6772d1b5fc51814b20cbc5e57dce8d58c8896b00226a4e94c05dd4e6
VerifyScript: PASS
HTTP: 200 OK
c49a4a8c6772d1b5fc51814b20cbc5e57dce8d58c8896b00226a4e94c05dd4e6
Saved: C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1\transactions.py
Explorer: https://blockstream.info/testnet/tx/c49a4a8c6772d1b5fc51814b20cbc5e57dce8d58c8896b00226a4e94c05dd4e6
Broadcast accepted. Wait for Confirmed before taking the final explorer screenshot.
PS C:\Users\pointzu\Desktop\大三上\区块链基础及应用\实验\Ex1> |
```

图 5: Ex1 发币终端结果：输出 13900 sat，手续费 1000 sat，广播成功

本次实际发币交易哈希为：

`c49a4a8c6772d1b5fc51814b20cbc5e57dce8d58c8896b00226a4e94c05dd4e6`

检查项	结果
交易输入	分币交易的第 0 号输出，14900 sat
收款输出	课程地址，13900 sat (0.000139 BTC)
手续费	1000 sat (0.00001 BTC)
实际大小与费率	192 bytes，约 5.21 sat/vB
本地验证	VerifyScript: PASS
广播返回	HTTP: 200 OK，返回哈希与本地交易哈希一致
链上结果	已确认，区块 5151332

发币输出仅有一份，输入与输出间的 1000 sat 差额全部作为手续费。其他九个分币输出没有被本次发币交易引用。

3.6 发币链上确认截图

交易 c49a4a8c6772d1b5fc51814b20cbc5e57dce8d58c8896b00226a4e94c05dd4e6 确认1

时间戳: 2026-09-28 04:29:38 (模拟) | 手续费: 1,000 US\$0.00

已确认: 2小时之后 | 费率: 5.21 聪/字节

交易特征: SegWitz Taproot RBF

输入与输出 显示图表 明细

输入: mzT3NuHTiGLvKHDzJyN4NtLogEFLv3A2 (0.00014900 BTC)

ScriptSig (ASM): `Op_return 72 3045022100dc3369c509da309aef257bc946a374a9a7200ab3778ba32d545ab0d4145b7a02207f02f13d03576ca26413a3283aaf36a56dc39c37d3d440ca4f92f8a3978c0b501`

ScriptSig (HEX): `463048022100dc3369c509da309aef257bc946a374a9a7200ab3778ba32d545ab0d4145b7a02207f02f13d03576ca26413a3283aaf36a56dc39c37d3d440ca4f92f8a3978c0b501`

nSequence: 0xffffffff

上一次输出脚本: `Op_return 60`

之前输出类型: r2xx

输出: mv4myY35u5gjcDNzbMLKBQk8icCHuHfB (0.00013900 BTC)

ScriptPubKey (ASM): `Op_return`

ScriptPubKey (HEX): `7fa2142f5e7ab5650c0caa03983a77c9c3df9e62cb3fa28ac`

类型: r2xx

明细

大小	192 B	版本	1
虚拟大小	192 vB	锁定时间	0
调整后的虚拟大小	192 vB	签名操作	4
权重	768 wu	交易源数据	🔗

图 6: 发币交易已确认, 输入 0.000149 BTC, 向课程地址输出 0.000139 BTC

截图显示发币交易获得确认。最终公开 API 核对结果为 `confirmed=true`, 确认区块 **5151332**, 与浏览器截图一致。网页中的输入解锁脚本及输出锁定脚本也对应本次 P2PKH 交易。

4 结果汇总与分析

阶段	本人输入/sat	输出总额/sat	手续费/sat	确认区块
领取	—	150524	—	5148688
分币	150524	149000	1524	5151331
发币	14900	13900	1000	5151332

领取行只列出水龙头交易中属于本人的输出；水龙头自身的输入、其他输出及其手续费不计入本人的实验花费。分币和发币两笔交易的手续费合计为 $1524 + 1000 = 2524$ sat。本人保留九份分币输出，共 $9 \times 14900 = 134100$ sat，课程地址收到 13900 sat：

$$150524 = 134100 + 13900 + 2524 \quad (\text{sat}).$$

这说明资金流向与手续费满足守恒关系。三笔交易的完整哈希保存在 `transactions.py`，最终查询记录保存至 `chain_status.txt`。

4.1 遇到的问题与解决方法

1. **DLL 加载失败**。实际问题是加载了其他软件的不匹配 DLL；让当前进程优先找到课程目录中已验证的 64 位 DLL 后恢复正常，无需修改系统全局配置。
2. **旧查询接口找不到交易**。BlockCypher 未查到领取记录，改用助教提示的 testnet3 浏览器核对，并使用 Esplora 广播接口；不能因一个浏览器查不到就断定资金未到账。
3. **输出编号容易误填**。依据交易输出列表确定领取输出为 1、分币后使用输出为 0。
4. **浮点金额不适合交易输出**。使用 `Decimal` 计算并转为整数 sat，同时检查输出总额小于输入，留出手续费。
5. **广播与确认混淆**。广播后先显示未确认，等待矿工打包后再保存确认截图；同一输出不能因等待而重复花费。

5 实验总结

本次实验完成测试币领取记录核对、十输出分币、P2PKH 脚本编写、交易签名与广播，并确认两笔新交易已入块。通过实际操作，理解了 UTXO 的定位和整体花费规则、公钥哈希与签名的双重验证，以及手续费由输入输出差额形成的机制。

数字签名解决花费授权问题，节点对 UTXO 和交易规则的验证及区块确认共同维护资金记录的一致性。本地脚本验证、接口接受和链上确认是不同阶段。

参考资料

1. 课程 Ex1.pdf
2. Blockstream Esplora API: <https://github.com/Blockstream/esplora/blob/master/API.md>。
3. 本次实验的 `transactions.py`、运行输出、截图与最终链上查询记录。